

MATERI PEMBELAJARAN

Tipe Data Dasar

dalam Bahasa Pemrograman C++

Pemrograman C++ | Tingkat Dasar

Modul ini membahas tipe data dasar (fundamental/primitive) yang tersedia dalam bahasa C++. Memahami tipe data adalah fondasi utama sebelum mempelajari struktur program yang lebih kompleks. Setiap tipe data menentukan jenis nilai yang dapat disimpan, ukuran memori, dan operasi yang dapat dilakukan.

1. Apa Itu Tipe Data?

Tipe data adalah klasifikasi yang memberi tahu compiler jenis nilai apa yang akan disimpan oleh sebuah variabel. Dalam C++, setiap variabel **wajib** dideklarasikan dengan tipe data tertentu sebelum digunakan.

- Menentukan **ukuran memori** yang dialokasikan
- Menentukan **rentang nilai** yang dapat disimpan
- Menentukan **operasi** yang valid pada variabel tersebut

C++ membagi tipe data menjadi tiga kelompok besar:

Kelompok	Contoh	Keterangan
Tipe Dasar (Fundamental)	int, float, char, bool	Tipe bawaan bahasa C++
Tipe Turunan (Derived)	array, pointer, reference	Dibentuk dari tipe dasar
Tipe Buatan Pengguna	struct, class, enum	Didefinisikan programmer

2. Tipe Integer (Bilangan Bulat)

Tipe integer digunakan untuk menyimpan bilangan bulat (tanpa desimal). C++ menyediakan beberapa varian integer dengan ukuran dan rentang berbeda.

Tabel Tipe Integer:

Tipe	Ukuran	Rentang Nilai	Penggunaan
short	2 byte	-32,768 s/d 32,767	Nilai kecil, hemat memori
int	4 byte	-2,147,483,648 s/d 2,147,483,647	Umum digunakan
long	4/8 byte	Bergantung platform	Nilai lebih besar dari int
long long	8 byte	-9.2×10^{18} s/d 9.2×10^{18}	Nilai sangat besar
unsigned int	4 byte	0 s/d 4,294,967,295	Hanya nilai positif
unsigned short	2 byte	0 s/d 65,535	Positif, hemat memori

Contoh Program:

```
#include <iostream>
using namespace std;

int main() {
    short nilaiKecil = 100;
    int umur = 20;
    long populasi = 2700000000L;
    long long jarak = 9460730472580800LL; // jarak 1 tahun cahaya (meter)
    unsigned int stok = 4000000000U;

    cout << "Short : " << nilaiKecil << endl;
    cout << "Int : " << umur << endl;
    cout << "Long : " << populasi << endl;
    cout << "Long long: " << jarak << endl;
    cout << "Unsigned : " << stok << endl;
    return 0;
}
```

■ Gunakan suffix L untuk long, LL untuk long long, U untuk unsigned agar compiler tidak salah interpretasi.

3. Tipe Floating Point (Bilangan Desimal)

Tipe floating point digunakan untuk menyimpan bilangan yang memiliki bagian desimal. C++ menyediakan tiga presisi yang berbeda.

Tipe	Ukuran	Presisi	Rentang Approx.	Suffix
float	4 byte	~7 digit	$\pm 3.4 \times 10^{\pm 38}$	f (contoh: 3.14f)
double	8 byte	~15 digit	$\pm 1.7 \times 10^{\pm 308}$	(tidak perlu suffix)
long double	12/16 byte	~18-19 digit	$\pm 1.18 \times 10^{\pm 4932}$	L (contoh: 3.14L)

Contoh Program:

```
#include <iostream>
#include <iomanip> // untuk setprecision
using namespace std;

int main() {
    float nilaiFloat = 3.14159f;
    double nilaiDouble = 3.14159265358979;
    long double nilaiLD = 3.14159265358979323846L;

    cout << fixed << setprecision(10);
    cout << "Float : " << nilaiFloat << endl;
    cout << "Double : " << nilaiDouble << endl;
    cout << "Long double: " << nilaiLD << endl;
    return 0;
}
```

■ Untuk perhitungan keuangan atau ilmiah yang butuh presisi tinggi, gunakan double. Hindari float untuk perhitungan kritis karena presisinya terbatas.

4. Tipe Character (Karakter)

Tipe **char** digunakan untuk menyimpan satu karakter tunggal. Secara internal, char menyimpan nilai integer (kode ASCII) dari karakter tersebut.

Tipe	Ukuran	Rentang	Keterangan
char	1 byte	-128 s/d 127	Karakter ASCII standar
unsigned char	1 byte	0 s/d 255	Karakter ASCII diperluas
wchar_t	2/4 byte	Unicode	Karakter Unicode/wide
char16_t	2 byte	UTF-16	Unicode 16-bit (C++11)
char32_t	4 byte	UTF-32	Unicode 32-bit (C++11)

Contoh Program:

```
#include <iostream>
using namespace std;

int main() {
    char huruf = 'A';
    char angka = '7'; // '7' berbeda dengan integer 7!
    char spesial = '\n'; // newline character

    cout << "Karakter : " << huruf << endl;
    cout << "Nilai ASCII 'A': " << (int)huruf << endl; // 65

    // Aritmatika pada char
    char berikutnya = huruf + 1; // 'B'
    cout << "Karakter berikutnya: " << berikutnya << endl;
    return 0;
}
```

■ Gunakan tanda petik tunggal (') untuk karakter, dan tanda petik ganda (") untuk string. Keduanya berbeda tipe!

5. Tipe Boolean

Tipe **bool** hanya menyimpan dua nilai: **true** (benar) atau **false** (salah). Tipe ini sangat penting dalam logika percabangan dan perulangan.

Tipe	Ukuran	Nilai Valid	Representasi Integer
bool	1 byte	true / false	true = 1, false = 0

Contoh Program:

```
#include <iostream>
using namespace std;

int main() {
    bool lulus = true;
    bool gagal = false;
    bool hasilCek = (10 > 5); // true

    cout << boolalpha; // tampilkan 'true'/'false', bukan 1/0
    cout << "Lulus : " << lulus << endl;
    cout << "Gagal : " << gagal << endl;
    cout << "10 > 5 : " << hasilCek << endl;

    // Tanpa boolalpha
    cout << noboolalpha;
    cout << "Nilai int true : " << (int)lulus << endl; // 1
    return 0;
}
```

6. Tipe Void

Tipe **void** berarti "tidak ada nilai". Berbeda dengan tipe lain, void tidak dapat digunakan untuk mendeklarasikan variabel biasa. Penggunaannya terbatas pada:

- Fungsi yang **tidak mengembalikan nilai**: `void cetakHalo() { ... }`
- Pointer generik yang dapat menunjuk ke tipe data apapun: `void* ptr;`

```
#include <iostream>
using namespace std;

void cetakPesan(string pesan) { // void: tidak return nilai
    cout << pesan << endl;
}

int main() {
    cetakPesan("Halo, C++!");

    int x = 42;
    void* ptr = &x; // void pointer: tipe generik
    cout << "Nilai x: " << *(int*)ptr << endl;
    return 0;
}
```

7. Modifier Tipe Data

Modifier mengubah sifat tipe data dasar. C++ menyediakan empat modifier:

Modifier	Fungsi	Contoh Penggunaan
signed	Dapat menyimpan nilai negatif & positif (default)	signed int x = -5;
unsigned	Hanya nilai positif (rentang lebih besar)	unsigned int y = 50000;
short	Mengurangi ukuran memori	short int z = 100;
long	Menambah ukuran/rentang nilai	long int w = 1000000L;

■ Modifier dapat dikombinasikan, contoh: *unsigned long long int* untuk nilai bulat positif sangat besar (0 s/d $\sim 1.8 \times 10^{19}$).

8. Mengetahui Ukuran & Batas Tipe Data

Gunakan operator **sizeof()** untuk mengetahui ukuran memori suatu tipe, dan header **<climits>** / **<cfloat>** untuk batas nilainya.

```
#include <iostream>
#include <climits> // batas integer
#include <cfloat> // batas floating point
using namespace std;

int main() {
    // Ukuran memori
    cout << "--- Ukuran Memori ---" << endl;
    cout << "char : " << sizeof(char) << " byte" << endl;
    cout << "int : " << sizeof(int) << " byte" << endl;
    cout << "float : " << sizeof(float) << " byte" << endl;
    cout << "double : " << sizeof(double) << " byte" << endl;
    cout << "long long : " << sizeof(long long) << " byte" << endl;

    // Batas nilai
    cout << "\n--- Batas Nilai ---" << endl;
    cout << "INT_MIN : " << INT_MIN << endl;
    cout << "INT_MAX : " << INT_MAX << endl;
    cout << "FLT_MAX : " << FLT_MAX << endl;
    cout << "DBL_MAX : " << DBL_MAX << endl;
    return 0;
}
```

9. Konversi Tipe Data (Type Casting)

Konversi tipe adalah proses mengubah nilai dari satu tipe ke tipe lain. C++ mendukung dua jenis konversi:

a) Konversi Implisit (Otomatis)

Dilakukan otomatis oleh compiler ketika mengkonversi ke tipe yang lebih besar (misalnya int ke double). Disebut juga *widening conversion*.

```
int angka = 10;
double hasil = angka; // int otomatis ke double → 10.0
float desimal = 3.7f;
double presisi = desimal; // float ke double (aman)
```

b) Konversi Eksplisit (Manual / Cast)

Dilakukan programmer secara manual. Bisa menyebabkan kehilangan data jika konversi ke tipe yang lebih kecil (*narrowing conversion*).

```
double pi = 3.14159;
int bulat = (int)pi; // C-style cast → 3 (bagian desimal hilang!)
int bulat2 = static_cast<int>(pi); // C++ style cast (lebih aman)

char huruf = (char)65; // 65 → 'A'
int kode = (int)'Z'; // 'Z' → 90
```

■ Selalu gunakan `static_cast<>` dalam C++ modern karena lebih aman dan eksplisit dibanding C-style cast (tipe). `static_cast` memberikan error compile-time jika konversi tidak masuk akal.

10. Ringkasan & Panduan Memilih Tipe Data

Gunakan tabel berikut sebagai panduan cepat memilih tipe data yang tepat:

Kebutuhan	Tipe yang Direkomendasikan
Bilangan bulat umum	int
Bilangan bulat sangat besar	long long
Bilangan bulat hanya positif	unsigned int / unsigned long long
Bilangan desimal (akurasi normal)	double
Bilangan desimal (hemat memori)	float
Satu karakter	char
Nilai benar/salah	bool
Fungsi tanpa nilai kembali	void
Hemat memori (integer kecil)	short

Tips Penting: Selalu pilih tipe data yang paling sesuai kebutuhan. Menggunakan tipe yang terlalu besar memboroskan memori, sedangkan tipe yang terlalu kecil bisa menyebabkan *overflow* (nilai melampaui batas).

Selamat belajar C++!

Materi ini dibuat untuk keperluan pembelajaran bahasa pemrograman C++.